Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Report for Project 4

Due date: 27 April 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Computing π with MPI [10 points]

1.1. Parellization using MPI

The parellel implementation of the π computation is based on the idea of dividing the interval $[0, 1]$ into N subintervals, where N is the number of processes. Each process computes the contribution to π from its assigned subinterval and then all contributions are summed together using MPI. In listing 1 we show the implementation of the parallel π computation using MPI.

Listing 1: Parallel π computation using MPI

```
MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);

h = 1.0 / (double)N;

MPI_Barrier(MPI_COMM_WORLD);
time_start = walltime();

for (uint64_t i = (uint64_t)rank; i < N; i += (uint64_t)size) {
    double x = (i + 0.5) * h;
    local_sum += 4.0 / (1.0 + x * x);
}

MPI_Reduce(&local_sum, &sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
double time = walltime() - time_start;
if (rank == 0) {
    pi = sum * h;
    printf("# pi N procs time [secs]\n");
    printf("%.15f %20" PRIu64 " %3d %.8f\n", pi, N, size, time);
    }

MPI_Finalize();
```

Each process starts from its rank and iterates over the indices of the subintervals assigned to it by incrementing the index by the total number of processes in each iteration. This way, each process computes the contribution to π from its assigned subintervals without overlap. After the local sums are computed, they are reduced to a single sum using `MPI_Reduce`, which is then used to compute the final value of π on the root process (rank 0).

1.2. Strong scaling

To evaluate the strong scaling of the parallel π computation, we run the program with a fixed problem size N and varying number of processes. The results are shown in figure 1, where we plot the speedup achieved by the parallel implementation compared to the serial version.

Table 1: Strong scaling results using the OpenMP reduction clause ($N = 10^9$).

Threads	Speedup
1	1.00
2	1.84
4	3.26
8	6.39
16	8.86
32	17.66
64	31.06

The strong scaling results obtained with MPI show a near-linear increase in speedup for a small number of processes, closely following the ideal trend. As the number of processes increases, the measured speedup progressively deviates from the ideal curve, highlighting the impact of communication overhead and reduced computational workload per process. Nevertheless, the scalability remains good, reaching a speedup close

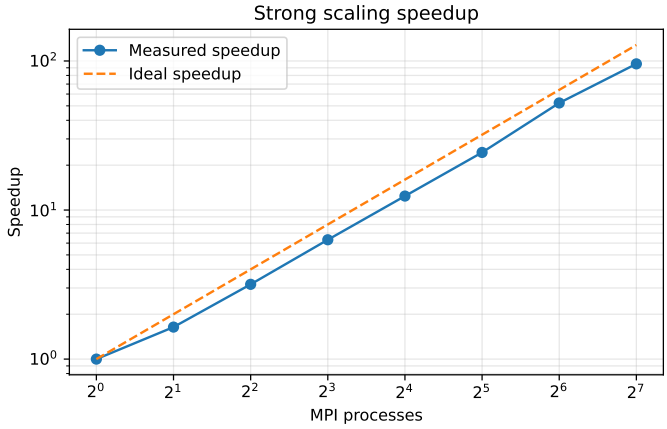


Figure 1: Strong scaling speedup of the parallel π computation. Obtained with $N = 10^9$.

to 100 with 128 processes.

Comparing these results with the OpenMP reduction implementation in Table 1, we observe a very similar scaling behavior. Both approaches exhibit efficient parallelization at low to moderate parallelism and a loss of efficiency at higher core counts. However, OpenMP achieves slightly better efficiency at comparable core counts, likely due to the shared-memory model which avoids explicit communication overhead.

1.3. Weak scaling

The results of the weak scaling tests are shown in figure 2, where we plot the weak scaling efficiency as a function of the number of processes.

The weak scaling efficiency remains high across all tested configurations, staying close to the ideal value of 1. This indicates that the workload is well balanced among MPI processes and that the parallelization strategy is effective.

A slight degradation in efficiency is observed as the number of processes increases, particularly at the highest core counts. This behavior is expected and can be attributed to increasing communication overhead and synchronization costs, which become more significant as more processes are involved.

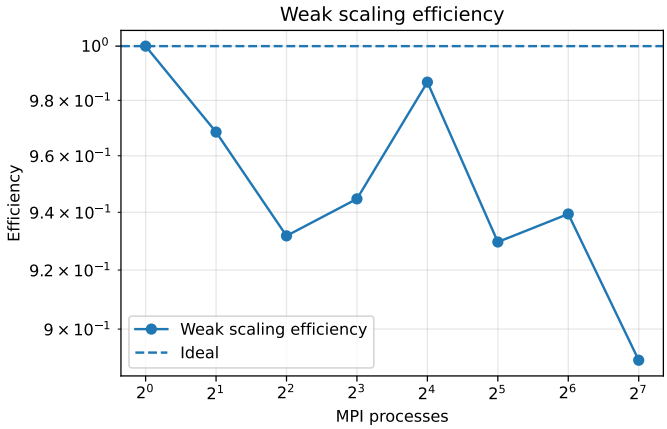


Figure 2: Weak scaling efficiency of the parallel π computation.

2. Ring sum using MPI [10 points]

In listing 2 we show the implementation of the ring sum using MPI.

Listing 2: Ring sum using MPI

```
int size, rank;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

// IMPLEMENT: Ring sum algorithm
int sum = rank;
for (int i = 0; i < size - 1; i++) {
  int send_to = (rank + 1) % size;
  int recv_from = (rank - 1 + size) % size;

  int recv_sum;
  MPI_Sendrecv(&sum, 1, MPI_INT, send_to, 0,
               &recv_sum, 1, MPI_INT, recv_from, 0,
               MPI_COMM_WORLD, MPI_STATUS_IGNORE);

  sum += recv_sum;
}
printf("Process %i: Sum = %i\n", rank, sum);

// Finalize MPI
MPI_Finalize();
```

The implementation uses a **ring communication pattern**, where each process exchanges data only with its immediate neighbors in a cyclic topology. Specifically, process i sends its current partial sum to process $(i+1) \bmod P$ and receives a value from process $(i-1+P) \bmod P$, where P is the total number of processes.

Each process initializes its local sum with its rank and performs $P-1$ communication steps. At every iteration, it forwards its current partial sum to the next process and accumulates the value received from the previous one. This ensures that, after all iterations, every process has received contributions from all other processes and therefore holds the global sum.

To avoid deadlock issues, the implementation uses the `MPI_Sendrecv` routine instead of separate blocking send and receive calls. This function performs the send and receive operations in a single, coordinated step, ensuring that communication progresses safely without requiring explicit ordering between processes.

3. Cartesian domain decomposition and ghost cells exchange [20 points]

3.1. Cartesian 2D communicator

In listing 3 we show the implementation of the Cartesian 2D communicator using MPI.

Listing 3: Cartesian 2D communicator using MPI

```
1  int rank, size, i, j, dims[2], periods[2], rank_north, rank_south,
       rank_west, rank_east;
2  double data[DOMAINSIZE*DOMAINSIZE];
3  MPI_Request request;
4  MPI_Status status;
5  MPI_Comm comm_cart;
6  MPI_Datatype data_ghost;
7
8  dims[0]= 3;
9  dims[1]= 4;
10 periods[0]= 1;
11 periods[1]= 1;
12
13 MPI_Cart_create(MPI_COMM_WORLD, 2 ,dims , periods , 0 , &comm_cart);
14
15 MPI_Cart_shift(comm_cart, 1, 1, &rank_south, &rank_north);
16 MPI_Cart_shift(comm_cart, 0, 1, &rank_west, &rank_east);
```

The code first defines the Cartesian topology of the process grid by setting `dims[0] = 3` and `dims[1] = 4`, corresponding to a 3×4 decomposition. The array `periods` is set to 1 in both directions, meaning that the topology is periodic both horizontally and vertically: processes at one boundary are connected to processes on the opposite boundary. This implements the cyclic communication pattern required by the exercise.

The call to `MPI_Cart_create` constructs a new communicator, `comm_cart`, in which all processes are arranged according to this two-dimensional Cartesian topology. Using a Cartesian communicator makes neighbor discovery much simpler than manually computing ranks.

The two calls to `MPI_Cart_shift` are then used to determine the ranks of the neighboring processes in the two coordinate directions. In particular, the first call identifies the south and north neighbors, while the second call identifies the west and east neighbors.

3.2. Derived data type for column boundary

In listing 4 we show the implementation of the derived data type for column boundary using MPI.

Listing 4: Derived data type for column boundary using MPI

```
1 MPI_Type_vector(SUBDOMAIN, 1, DOMAINSIZE, MPI_DOUBLE, &data_ghost);
2 MPI_Type_commit(&data_ghost);
```

The ghost-cell exchange in the horizontal direction involves non-contiguous memory, because a column of the local two-dimensional array is stored with a stride equal to `DOMAINSIZE`. To communicate such a column efficiently, a derived MPI datatype is created with `MPI_Type_vector`.

This datatype describes `SUBDOMAIN` elements of type `MPI_DOUBLE`, with one element taken from each row and a spacing of `DOMAINSIZE` elements between successive entries. In this way, the datatype represents exactly one column of the local subdomain. After its definition, `MPI_Type_commit` makes the datatype available for communication.

3.3. Exchange ghost cells

In listing 5 we show the implementation of the ghost cells exchange using MPI.

Listing 5: Ghost cells exchange using MPI

```
/* send top interior row (j = SUBDOMAIN) to north, receive bottom ghost
   row (j = 0) from south */
    MPI_Sendrecv(&data[SUBDOMAIN + 1 * DOMAINSIZE], 1, data_ghost,
                 rank_north, 0,
                 &data[0 + 1 * DOMAINSIZE], 1, data_ghost,
                 rank_south, 0,
                 comm_cart, MPI_STATUS_IGNORE);

    /* send bottom interior row (j = 1) to south,
       receive top ghost row (j = SUBDOMAIN+1) from north */
    MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], 1, data_ghost,
                 rank_south, 1,
                 &data[(SUBDOMAIN + 1) + 1 * DOMAINSIZE], 1, data_ghost,
                 rank_north, 1,
                 comm_cart, MPI_STATUS_IGNORE);


    /* send right interior column (i = SUBDOMAIN) to east,
       receive left ghost column (i = 0) from west */
    MPI_Sendrecv(&data[1 + SUBDOMAIN * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
                 rank_east, 2,
                 &data[1 + 0 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
                 rank_west, 2,
                 comm_cart, MPI_STATUS_IGNORE);

    /* send left interior column (i = 1) to west,
       receive right ghost column (i = SUBDOMAIN+1) from east */
    MPI_Sendrecv(&data[1 + 1 * DOMAINSIZE], SUBDOMAIN, MPI_DOUBLE,
                 rank_west, 3,
                 &data[1 + (SUBDOMAIN + 1) * DOMAINSIZE], SUBDOMAIN,
                    MPI_DOUBLE,
                 rank_east, 3,
                 comm_cart, MPI_STATUS_IGNORE);
```

This part of the code performs the exchange of ghost cells with the four neighboring processes of the Cartesian grid. The communication is split into four steps, one for each boundary: north, south, east, and west. In each case, the interior boundary of the local subdomain is sent to the corresponding neighbor, while the matching ghost layer is received from the opposite side.

The first two `MPI_Sendrecv` calls handle the exchange in the vertical direction. The top interior boundary is sent to the north neighbor and the bottom ghost layer is received from the south neighbor. Similarly, the bottom interior boundary is sent to the south neighbor and the top ghost layer is received from the north neighbor. Because the domain is periodic, these communications also work correctly for processes located at the physical boundaries of the global domain.

The last two `MPI_Sendrecv` calls handle the exchange in the horizontal direction. Here the communication involves columns, so the memory layout must be treated carefully. One boundary column is exchanged with the east and west neighbors, and the received values are stored in the corresponding ghost columns.

The use of `MPI_Sendrecv` is particularly important because it avoids possible deadlock issues that could arise if all processes tried to perform blocking send operations before posting matching receives. By combining send and receive into one single communication call, the ghost-cell exchange is both safe and simple to implement.

The output is shown in listing 6.

Listing 6: Output of ghost cell exchange

```
North rank 6, South rank 4, West rank 1, East rank 9
```

```
data of rank 5 after communication
j
^
| -1.0  6.0  6.0  6.0  6.0  6.0  6.0 -1.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
|  1.0  5.0  5.0  5.0  5.0  5.0  5.0  9.0
| -1.0  4.0  4.0  4.0  4.0  4.0  4.0 -1.0
----------------------------------------> i
```

4. Parallelizing the Mandelbrot set using MPI [30 points]

5. Parallel matrix-vector multiplication and the power method [30 points]